

VESTRA

Operations & Monitoring Guide

Version 3.0.0 — June 2026

AI-Powered Property Trust Platform for Africa

Backend API	http://localhost:8000
Swagger Docs	http://localhost:8000/docs
Health Check	http://localhost:8000/health
Prometheus Metrics	http://localhost:8000/metrics
Frontend	http://localhost:3000
Vestra Website	https://vestra.co.ke

Table of Contents

1. Quick Start — Running the System
2. Architecture Overview
3. Backend Deep Dive
4. Monitoring & Health Checks
5. Prometheus Metrics Reference
6. Logging & Observability
7. Database Operations
8. Redis Caching Layer
9. Security Architecture
10. API Route Reference (22 modules, 85+ endpoints)
11. Environment Configuration
12. Docker Deployment
13. Troubleshooting
14. Demo Credentials & Seed Data

1. Quick Start — Running the System

Option A: PowerShell Startup Scripts (Windows)

From the **vestra/** directory, run:

```
./start-all.ps1
```

This launches both backend (port 8000) and frontend (port 3000) in separate minimized PowerShell windows.

Option B: Manual Start

Backend (Terminal 1):

```
cd vestra/backend venv/Scripts/python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend (Terminal 2):

```
cd vestra/frontend-build npm run dev
```

Option C: Docker (Production)

```
export POSTGRES_PASSWORD=secure_password export REDIS_PASSWORD=secure_redis export SECRET_KEY=$(openssl rand -base64 64) docker-compose up -d
```

Verification

After starting, verify the system is healthy:

```
# Backend health check curl http://localhost:8000/health # Should return {"status":"healthy","database":"connected","redis":"connected"}
```

Stop the System

```
./stop-all.ps1
```

2. Architecture Overview

Vestra is an AI-powered property trust platform with a FastAPI backend, Next.js 16 frontend, PostgreSQL 16 database, and Redis 7 caching layer. The AI engine is entirely rule-based (no external API calls), providing sub-10ms fraud detection, trust scoring, and property valuation.

Layer	Technology	Port	Purpose
Frontend	Next.js 16 + React 19.2 + TypeScript	3000	PWA with offline support
API Gateway	FastAPI 0.111 + Uvicorn	8000	REST API + WebSocket
AI Engine	Built-in rule-based (Python)	—	7 sub-engines, zero external APIs
Database	PostgreSQL 16 (asyncpg)	5432	6 tables, 30+ indexes, FTS
Cache	Redis 7 (redis-py)	6379	6-layer caching, rate limiting
Payments	M-Pesa Daraja + Stripe	—	STK Push, B2C, webhooks
Messaging	WhatsApp Business API	—	Chat + notifications
Monitoring	Prometheus + Structured Logging	8000	/metrics endpoint

Request Flow (Example: AI Property Search)

1. User submits search → Next.js calls GET /api/properties/ai-search?q=...
2. SecurityHeadersMiddleware adds CSP/HSTS/X-Frame headers
3. RateLimitMiddleware checks Redis sliding window (120 req/min)
4. GzipCompressionMiddleware compresses response >1KB
5. RequestLoggingMiddleware assigns correlation ID + logs timing
6. Route handler calls VestraAI search parser → structured filters
7. search_properties() → Redis cache check (2-min TTL) → PostgreSQL FTS
8. GIN index scan → relevance-ranked results → JSON response
9. Response headers: X-Correlation-ID, X-Response-Time-Ms, X-RateLimit-Remaining

3. Backend Deep Dive

Middleware Stack (applied in order)

#	Middleware	Purpose
1	SecurityHeadersMiddleware	CSP, HSTS, X-Frame-Options, X-XSS-Protection, Permissions-Policy
2	RequestSizeLimitMiddleware	Rejects requests >5MB with 413
3	RateLimitMiddleware	Redis sliding-window: Auth 10/min, General 120/min, Admin 300/min
4	GzipCompressionMiddleware	Compresses text/json >1KB (gzip level 6)
5	metrics_middleware	Prometheus: REQUEST_COUNT, REQUEST_LATENCY, REQUEST_IN_FLIGHT
6	RequestLoggingMiddleware	JSON logs: correlation_id, method, path, status, duration_ms, IP
7	CORSMiddleware	Allow origins from settings, expose rate-limit/correlation headers

AI Engine — 7 Sub-Engines

Engine	Function	Latency
FraudDetector	Keyword scan (23 keywords), price sanity, document check, agent verification	<10ms
TrustEngine	Composite 0-100 with ownership confidence + recommendation	<5ms
PriceAnalyser	City price band comparison, bedroom adjustment, 20% tolerance	<5ms
SearchParser	Natural language → structured filters (city, bedrooms, price, type)	<10ms
DocumentAnalyser	Checks submitted docs against required list per listing type	<5ms
ValuationEngine	Size × sqft price, age depreciation, amenity bonus, investment score	<10ms
MarketIntelligence	City-level trends, supply/demand, best time to buy, investor tips	<5ms

4. Monitoring & Health Checks

Health Endpoints

Endpoint	Method	Purpose	Response Example
/health	GET	Full health (DB + Redis check)	{"status":"healthy","database":"connected","redis":"connected"}
/health/live	GET	K8s liveness probe (minimal)	{"status":"alive"}
/health/ready	GET	K8s readiness probe (DB check)	{"status":"ready"}
/metrics	GET	Prometheus metrics endpoint	vestra_http_requests_total{...} 1234
/	GET	Root info (version, env)	{"name":"Vestra","version":"3.0.0","status":"operational"}
/api/monitoring/health/full	GET	Detailed health (auth required)	Full system health + services
/api/monitoring/health/services	GET	Service status breakdown	DB, Redis, M-Pesa, Stripe, WhatsApp
/api/monitoring/health/resources	GET	CPU, memory, disk usage	System resource metrics
/api/monitoring/health/database	GET	DB pool stats, query times	Connection pool metrics
/api/monitoring/health/redis	GET	Redis memory, keys, hits	Cache hit rate, memory usage

Health Check Responses Reference

Healthy: status = 'healthy', database = 'connected', redis = 'connected'

Degraded: status = 'degraded', one dependency shows 'error: ...' or 'unavailable'

Down: /health/live returns non-200 or times out

Response Headers (Every API Response)

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	DENY	Prevent clickjacking
X-XSS-Protection	1; mode=block	XSS filter
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Enforce HTTPS (1 year)
Referrer-Policy	strict-origin-when-cross-origin	Privacy
Permissions-Policy	camera=(), microphone=(), geolocation=(self), push=(), sync-xr=(self)	Browser feature access
X-Correlation-ID	<uuid>	Request tracing across services
X-Response-Time-Ms	<ms>	Server processing time
X-RateLimit-Remaining	<count>	Requests remaining in window

5. Prometheus Metrics Reference

All metrics exposed at **GET /metrics** in Prometheus text format.

Metric Name	Type	Labels	Description
vestra_http_requests_total	Counter	method, endpoint, status	Total HTTP requests
vestra_http_request_duration_seconds	Histogram	method, endpoint	Request latency distribution
vestra_http_requests_in_flight	Gauge	—	Concurrent requests being processed
vestra_properties_listed_total	Counter	property_type, listing_type	Properties created
vestra_verifications_run_total	Counter	recommendation	AI verifications executed
vestra_payments_received_total	Counter	method, purpose	Payment count
vestra_payments_amount_kes_total	Counter	purpose	Revenue in KES
vestra_users_registered_total	Counter	role	Signup count by role
vestra_fraud_risk_score	Histogram	—	Fraud score distribution (0-100)
vestra_trust_score	Histogram	—	Trust score distribution (0-100)

Example Prometheus Configuration

```
scrape_configs: - job_name: 'vestra' scrape_interval: 15s static_configs: - targets: ['localhost:8000'] metrics_path: '/metrics'
```

6. Logging & Observability

All logs are structured JSON. Every request gets a unique `correlation_id` for tracing.

Log Format:

```
{"event": "user_registered", "user_id": 42, "correlation_id": "abc123"}
```

Key Events Logged:

- **startup** — App name, version, environment, Redis connection status
- **shutdown** — Graceful shutdown with resource cleanup
- **request** — Every HTTP request with method, path, status, `duration_ms`, `client_ip`
- **unhandled_error** — Full traceback for unexpected exceptions
- **user_registered / user_login** — Auth events
- **property_created / property_verified** — Property lifecycle
- **payment_initiated / payment_completed** — Payment flow
- **subscription_created / subscription_cancelled** — Subscription events
- **fraud_reported / fraud_reviewed** — Trust & safety
- **escrow_created / escrow_released** — Escrow lifecycle
- **dispute_created / dispute_resolved** — Dispute workflow

Slow Request Detection: Requests >1 second are logged at WARNING level.

Log Level: INFO in development, WARNING in production (configurable via `LOG_LEVEL` env var).

7. Database Operations

Connection Pool Configuration

Setting	Value	Description
DATABASE_POOL_SIZE	20	Base pool size
DATABASE_MAX_OVERFLOW	40	Extra connections above base
DATABASE_POOL_RECYCLE	3600s	Recycle connections after 1 hour
pool_pre_ping	True	Verify connections before use
pool_timeout	30s	Wait up to 30s for a connection
expire_on_commit	False	Prevents MissingGreenlet errors in async

Tables (7 core tables)

Table	Rows (seed)	Key Indexes
users	9	email UNIQUE, phone UNIQUE, role
properties	16	16 indexes incl. GIN FTS, city, county, price, status
agent_profiles	2	user_id UNIQUE, badge_level
documents	20+	property_id, uploader_id, document_type
verifications	17	property_id, status, trust_score
payments	21	user_id, status, checkout_request_id, reference UNIQUE
audit_logs	—	user_id, action, resource_type, created_at

8. Redis Caching Layer

Vestra uses 6 layers of caching for sub-10ms response times:

Layer	TTL	What's Cached
Browser (Service Worker)	Indefinite	Static assets (JS, CSS, images, fonts) — PWA offline
HTTP Cache Headers	Per-response	Cache-Control: no-store for API, ETags for static
Redis — Property Detail	300s (5 min)	Single property JSON, cache-aside pattern
Redis — Property Listings	120s (2 min)	Search results hashed by params
Redis — AI Valuation	600s (10 min)	Valuation results hashed by property attributes
Redis — Market Insights	1800s (30 min)	City-level data changes slowly
Redis — Admin Stats	120s (2 min)	Dashboard stats (15+ expensive queries cached)
DB Connection Pool	—	20+40 connections, pre-ping, recycle 3600s
PostgreSQL Shared Buffers	—	Frequently accessed data in RAM, index-only scans

Cache Key Schema

- vestra:cache:<prefix>:<sha256hash[:16]> — Cached function results
- vestra:ratelimit:<type>:<client_ip> — Rate limit windows
- vestra:refresh:<user_id>:<token_jti> — Refresh tokens
- vestra:email_verify:<token> — Email verification (24h TTL)
- vestra:pw_reset:<token> — Password reset (30min TTL)
- vestra:search:<sha256hash[:16]> — Search results (2min TTL)
- vestra:sub:<user_id> — User subscription (5min TTL)

9. Security Architecture

Threat	Protection
Brute Force	Redis rate limiting: 10 auth req/min per IP, 5-fail account lockout (15 min)
XSS	Content-Security-Policy headers, output encoding
Clickjacking	X-Frame-Options: DENY
MIME Sniffing	X-Content-Type-Options: nosniff
MITM	HSTS max-age=31536000; includeSubDomains; preload
CSRF	CSRF protection enabled in settings
SQL Injection	SQLAlchemy ORM parameterized queries exclusively
File Upload	Type validation + 10MB size limit + MIME type whitelist
User Enumeration	Forgot password returns same response regardless of email existence
DDoS	Rate limiting per endpoint type, 5MB request body size limit
API Key Theft	SHA-256 hashed storage, shown only once on creation
M-Pesa Callback	Safaricom IP whitelist + HMAC signature verification + replay protection
WhatsApp Webhook	HMAC-SHA256 signature verification on all messages

Auth: JWT (HS256) + bcrypt, 60-min access tokens, 7-day refresh tokens, 6 RBAC roles.

Production Requirements: SECRET_KEY ≥ 32 chars, REDIS_PASSWORD set, DATABASE_URL has ?ssl=require, TURNSTILE_SECRET_KEY set for CAPTCHA, DEBUG=False.

10. API Route Reference (22 Modules, 85+ Endpoints)

Module	Prefix	Endpoints	Auth
Auth (email)	/api/auth	8 (register, login, me, change-password, forgot-password, verify-email)	Mixed
OTP Auth	/api/auth/otp	3 (send-otp, verify-otp, login)	Mixed
Properties	/api/properties	9 (CRUD, AI search, FTS, my, publish, listing-favorites)	Mixed
Verification	/api/verify	6 (request, run AI, status, documents upload, admin review)	Mixed
Payments	/api/payments	4 (mpesa initiate, mpesa callback, status, my)	Mixed
Admin	/api/admin	12 (stats, users CRUD, properties, verifications, payments, audit, fraud, KYC)	Mixed
AI Routes	/api/ai	4 (valuate, market insights, search parse)	Bearer
WhatsApp	/api/whatsapp	7 (webhook, send text/card/report/payment, broadcast)	Mixed
Subscriptions	/api/subscriptions	4 (plans, my, subscribe, cancel)	Bearer
Rentals	/api/rentals	7 (collect, schedule, units, dashboard, payments, receipt)	Bearer
KYC	/api/kyc	4 (submit, status, admin pending, admin review)	Bearer
Notifications	/api/notifications	5 (list, unread-count, mark-read, mark-all-read, preferences)	Bearer

Messages	/api/messages	5 (send, conversations, conversation, mark-read, unread)	Bearer
Fraud	/api/fraud	4 (report, check-blacklist, admin pending, admin review)	Bearer
Favorites	/api/favorites	4 (add, remove, list, check)	Bearer
Reports	/api/reports	2 (verification report JSON + PDF download)	Bearer
Enterprise	/api/enterprise	6 (keys CRUD, usage, webhooks CRUD)	Bearer
Monitoring	/api/monitoring	5 (health full/services/resources/database/redshift)	Bearer
Escrow	/api/escrow	9 (CRUD, deposit, balance, release, cancel, dispute admin)	Bearer
Disputes	/api/disputes	8 (file, categories, my, detail, admin assign/resolve, stats)	Bearer
Reviews	/api/reviews	5 (write, subject, my, property stats, top agents)	Mixed
Payouts	/api/payouts	7 (request, my, detail, admin process/complete, stats)	Bearer
Coupons	/api/coupons	3 (validate, apply, admin list)	Bearer

11. Environment Configuration

All settings are in **vestra/backend/.env** with type-safe defaults in **app/core/config.py**.

Category	Variable	Default	Required in Prod?
Database	DATABASE_URL	postgresql+asyncpg://postgres:postgres@localhost:5432/vestra	Yes (with 5432 - require)
Database	DATABASE_POOL_SIZE	20	No
Database	DATABASE_MAX_OVERFLOW	40	No
Redis	REDIS_URL	redis://localhost:6379/0	YES
Redis	REDIS_PASSWORD	""	YES (production)
Auth	SECRET_KEY	change-me-...	YES (≥32 chars)
Auth	ACCESS_TOKEN_EXPIRE_MINUTES	60	No
M-Pesa	MPESA_CONSUMER_KEY	""	For M-Pesa
M-Pesa	MPESA_CONSUMER_SECRET	""	For M-Pesa
M-Pesa	MPESA_SHORTCODE	174379	For M-Pesa
M-Pesa	MPESA_PASSKEY	""	For M-Pesa
M-Pesa	MPESA_ENV	sandbox	Change to production
Stripe	STRIPE_SECRET_KEY	""	For Stripe
Email	SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD	""	For email
WhatsApp	WHATSAPP_PHONE_NUMBER_ID / ACCESS_TOKEN	""	For WhatsApp
CAPTCHA	TURNSTILE_SECRET_KEY / SITE_KEY	""	YES (production)
App	APP_NAME	Vestra	No
App	APP_VERSION	3.0.0	No
App	ENVIRONMENT	development	Set to production
App	CORS_ORIGINS	http://localhost:3000	Set to frontend URL
App	BASE_URL	http://localhost:3000	Set to public URL
Rate Limit	RATE_LIMIT_AUTH_PER_MINUTE	10	No
Rate Limit	RATE_LIMIT_GENERAL_PER_MINUTE	120	No
Rate Limit	RATE_LIMIT_ADMIN_PER_MINUTE	300	No
Lockout	ACCOUNT_LOCKOUT_MAX_ATTEMPTS	5	No
Lockout	ACCOUNT_LOCKOUT_DURATION_MINUTE	15	No

12. Docker Deployment

Full stack via **docker-compose.yml** — 4 services:

Service	Image	Resources	Port
postgres	postgres:16-alpine	512MB limit, data checksums	5432
redis	redis:7-alpine	256MB maxmemory, allkeys-lru, AOF	6379
backend	Custom (multi-stage)	1GB limit, 4 Gunicorn workers	8000
frontend	Custom (Next.js standalone)	—	3000

Deployment Platforms Supported:

- Fly.io (Nairobi region, 2 CPUs, 1GB)
- Render (blueprint: API + Frontend + PostgreSQL + Redis)
- Railway (2 replicas, Dockerfile)
- Self-hosted VPS (docker-compose)
- Vercel (frontend only, API proxy to backend)

Gunicorn Production Config:

workers = CPU × 2 + 1

worker_class = uvicorn.workers.UvicornWorker

max_requests = 10000 (with 1000 jitter)

timeout = 30s, graceful_timeout = 10s, keepalive = 5s

backlog = 2048, JSON access logs

13. Troubleshooting

Backend won't start

- Check PostgreSQL is running: `pg_isready`
- Check Redis is running: `redis-cli ping`
- Verify `.env` has correct `DATABASE_URL` and `REDIS_URL`
- Check port 8000 is free: `netstat -ano | findstr 8000`
- Run seed data: `python seed.py`

Frontend won't start

- Run `npm install` from `frontend-build/`
- Check `.env.local` has `NEXT_PUBLIC_API_URL=http://localhost:8000`
- Clear Next.js cache: `rm -rf .next`
- Check port 3000 is free

Redis connection errors

- Redis is optional — app works without it (caching/rate-limiting disabled)
- Start Redis: `redis-server` or Docker: `docker run -p 6379:6379 redis:7-alpine`

Rate limited (429 errors)

- Auth: 10 req/min, General: 120 req/min, Admin: 300 req/min
- Wait for the window to reset or increase limits in `.env`

M-Pesa payments failing

- Check `MPESA_ENV=sandbox` for testing
- Sandbox credentials: use Safaricom Daraja test credentials
- Verify callback URL is publicly accessible

Slow API responses

- Check `/health` — if Redis is down, all requests go to DB
- Check `/api/monitoring/health/database` for pool exhaustion
- Increase `DATABASE_POOL_SIZE` or `MAX_OVERFLOW` if needed
- Enable GIN index: run `ensure_fts_index()` from `search_service`

14. Demo Credentials & Seed Data

All demo users share password: **demo1234**

Name	Email	Role	Notes
Admin	admin@vestra.co.ke	super_admin	Full system access
Jane Muthoni	jane.muthoni@email.com	agent	Verified agent, gold badge
David Kamau	david.kamau@email.com	agent	Verified agent
Peter Omondi	peter.omondi@email.com	seller	3 properties listed
Faith Wanjiku	faith.wanjiku@email.com	seller	2 properties listed
Grace Akinyi	grace.akinyi@email.com	landlord	2 rental units
Samuel Njoroge	samuel.njoroge@email.com	buyer	Demo buyer account
Mary Wekesa	mary.wekesa@email.com	buyer	Demo buyer account

Seed Data (16 properties, 17 verifications, 21 payments, 20+ documents)

Load seed data: `cd vestra/backend && python seed.py`